

Assignment 1: Orchestration and contextualization using docker containers

Task 1. Introduction to Docker containers and DockerHub

Question 1:

These two concepts operate at different layers of the infrastructure stack. **Contextualization** happens at build time: it shapes a single container into something specific — installing the required packages, setting environment variables, and defining the startup command. The Dockerfile is the mechanism; the output is a self-contained, reproducible unit ready for one specific role. In Task 1, installing `sl` and registering its path in the image is exactly this.

Orchestration is a runtime concern that operates across multiple containers. It governs startup ordering, inter-service networking, failure recovery, and scaling. In Task 2, docker compose takes on this role by coordinating the Spark master and worker into a functioning cluster without manual intervention.

The short version: contextualization defines **what** a container is; orchestration defines **how** a group of containers behaves.

Question 2:

Contents of the Dockerfile:

- `FROM ubuntu:22.04` — base image; the starting point for the entire build
- `RUN apt-get update` — refreshes the package index
- `RUN apt-get -y upgrade` — applies available updates to installed packages
- `RUN apt-get -y install sl` — installs `sl`, a terminal animation of a steam locomotive
- `ENV PATH="${PATH}:/usr/games/"` — adds `/usr/games/` to `PATH` so `sl` is callable without its full path
- `CMD ["echo", "Data Engineering-II."]` — default batch-mode output; overridden when running interactively

docker run -it mycontainer/first:v1 bash:

The `-i` flag keeps stdin open; `-t` allocates a pseudo-TTY. Together they enable an interactive shell session inside the container. Passing `bash` overrides the default `CMD`, so instead of printing the `echo` message and exiting, the container drops into a root shell. From there, running `sl` confirms the package is correctly installed and on the `PATH` — the locomotive animation plays in the terminal.

docker ps:

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED    STATUS    PORTS                               NAMES
ff78cdae7832   sparkaio/first:v0 "sh -c '/usr/local/s.."  2 days ago Up 2 days                spark-cluster-spark-worker-1
bdfc6b7f3671   sparkaio/first:v0 "sh -c '/usr/local/s.."  2 days ago Up 2 days    0.0.0.0:7077->7077/tcp,
spark-cluster-spark-master-1
                                0.0.0.0:8080->8080/tcp
```

Lists all currently running containers. The output shows the Task 2 Spark cluster is active — `spark-cluster-spark-master-1` and `spark-cluster-spark-worker-1`, both using `sparkaio/first:v0` and up for two days. The master exposes port 7077 (RPC) and 8080 (web UI) to the host; the worker has no exposed ports since it communicates internally.

docker images:

```
$ docker images
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
sparkaiio/first:v0 ea58c8765b23  2.11GB      892MB         U
```

Lists locally stored images. sparkaiio/first:v0 (ID ea58c8765b23) is the only image present — 2.11GB on disk, 892MB of actual content. The gap between disk usage and content size is typical; it reflects how Docker stores layered image data.

docker stats:

```
$ docker stats --no-stream
CONTAINER ID  NAME                                CPU %  MEM USAGE / LIMIT  MEM %  NET I/O          BLOCK I/O  PIDS
ff78cdae7832  spark-cluster-spark-worker-1       0.14%  203.1MiB / 3.82GiB  5.19%  1.81MB / 16.2MB  5.3MB / 149MB  35
bdfc6b7f3671  spark-cluster-spark-master-1       0.25%  193.4MiB / 3.82GiB  4.94%  16.2MB / 1.8MB   10.7MB / 181MB  36
```

Shows live resource usage per container (CPU, memory, network I/O, block I/O, process count). The `--no-stream` flag takes a single snapshot rather than running continuously. The master uses slightly more CPU than the worker, consistent with it handling job scheduling and coordination. Both containers are within comfortable memory bounds on this host.

Question 3:

docker build reads a Dockerfile and produces a static, reusable image. It runs once and the result is stored locally (or pushed to a registry). No containers are created.

docker run instantiates a new container from an existing image. Each call creates a fresh, isolated environment. The image itself is not modified.

docker exec runs a command inside an **already-running** container. It does not create anything — it attaches to an existing process namespace. Useful for debugging live containers without restarting them.

Sequence in practice: build to produce the image, run to launch a container from it, exec to inspect or interact with it while it is running.

Question 4:

DockerHub is a cloud-hosted registry for container images — essentially a version-controlled store where images can be pushed, pulled, and shared. Its practical value is distribution: rather than rebuilding an image from scratch on each new machine, a single docker pull fetches an exact, reproducible environment from anywhere. It also hosts official base images (ubuntu, python, postgres, etc.) that most Dockerfiles depend on, making it an invisible but critical part of nearly every Docker workflow.

For collaborative work and reproducibility in course assignments, it means anyone can replicate the exact environment with one command rather than setting it up manually.

The publicly available container for this task: **secon01/first:v1**.

Question 5:

docker build operates on a single Dockerfile and produces one image. It has no awareness of other services, ports, or how containers might relate to each other at runtime.

docker compose works one level above that. It reads a YAML service definition and handles building images if needed, creating a shared network, enforcing startup ordering via `depends_on`, and mapping ports — all in one command. In Task 2, the Spark worker needs to resolve sparkmaster by hostname; this only works because docker compose places both containers on the same internal network automatically. docker build alone has no mechanism to enable this.

Question 6:

Multi-stage builds use multiple FROM statements in a single Dockerfile. The idea is to separate the build environment from the runtime environment. Build-time dependencies — compilers, package managers, test tools — stay confined to an earlier stage and are never included in the final image. Only what is explicitly COPY --from= across the stage boundary makes it in.

```
# Stage 1: builder - installs sl with all its dependencies
FROM ubuntu:22.04 AS builder
RUN apt-get update && \
    apt-get -y upgrade && \
    apt-get -y install sl

# Stage 2: runtime - clean base, only the binary and required libraries
FROM ubuntu:22.04
COPY --from=builder /usr/games/sl /usr/games/sl
COPY --from=builder /usr/lib/x86_64-linux-gnu/libncurses.so.6 \
    /usr/lib/x86_64-linux-gnu/libncurses.so.6
COPY --from=builder /usr/lib/x86_64-linux-gnu/libtinfo.so.6 \
    /usr/lib/x86_64-linux-gnu/libtinfo.so.6
ENV PATH="${PATH}:/usr/games/"
CMD ["echo", "Data Engineering-II."]
```

The result: mycontainer/first:v2 is **117MB** versus **264MB** for v1 — a reduction of over 55%. Beyond size, the runtime image no longer contains apt, the package cache, or any other build-time artifact. Smaller attack surface, faster pulls, and a clear separation between what was needed to build the image and what is needed to run it.

Task 2. Build a multi-container Apache Spark cluster using docker compose

Question 1:

The configuration defines two services, both using the sparkaio/first:v0 image. The spark-master service sets hostname: sparkmaster — this is what the worker uses to connect, so it cannot be arbitrary. The master starts its process with start-master.sh and exposes ports 7077 (Spark RPC) and 8080 (web UI) to the host. The spark-worker service uses depends_on to delay its startup until the master is running, then connects to it via spark://sparkmaster:7077. Both services set SPARK_NO_DAEMONIZE=true so the Spark processes stay in the foreground and keep the containers alive.

```
services:
  spark-master:
    image: sparkaio/first:v0
    hostname: sparkmaster
    environment:
      - SPARK_NO_DAEMONIZE=true
    command: sh -c "/usr/local/spark/sbin/start-master.sh && tail -f /dev/null"
    ports:
      - "8080:8080"
      - "7077:7077"

  spark-worker:
    image: sparkaio/first:v0
    depends_on:
      - spark-master
    environment:
      - SPARK_NO_DAEMONIZE=true
    command: sh -c "/usr/local/spark/sbin/start-slave.sh spark://sparkmaster:7077 && tail -f /dev/null"
```

Question 2:

The `docker-compose.yml` file uses **YAML** (YAML Ain't Markup Language) — a human-readable data serialisation format. The top-level key is `services`, under which each container is defined as a named block. Indentation determines hierarchy; nested keys like `image`, `hostname`, `ports`, `depends_on`, and `environment` are standard Compose fields. There are no types to declare and no brackets — the structure is inferred entirely from whitespace.

Question 3:

The core limitation is that **docker compose** is single-host only. All containers run on the same machine, which means no horizontal distribution, no cross-node failover, and no load balancing across physical infrastructure. Beyond that, it has no native autoscaling, no built-in health-check-driven restart logic beyond basic restart: always, and no rolling update mechanism. For a Spark cluster that needs to span multiple nodes or recover from hardware failure, these gaps matter. Tools like Kubernetes or Docker Swarm exist precisely to address them.

Task 3. Introduction to different orchestration and contextualization frameworks

1 - At scale, the difference between a system that runs reliably and one that requires constant manual intervention often comes down to how well contextualization and orchestration have been handled. Contextualization addresses a static problem: ensuring that each node in a distributed system arrives correctly configured — with the right runtime, environment variables, and startup behaviour — before it accepts any work. Without this, every new instance requires manual setup, which does not scale. Orchestration addresses the dynamic problem: coordinating those instances at runtime, absorbing failures, managing the network topology, and adapting capacity to demand without human involvement.

Several frameworks approach this problem space from different angles. Docker Compose sits at the simplest end: it brings up multiple containers on a single host from a YAML definition and handles internal networking and startup order. It is well-suited for development environments and simple deployments, but the single-host constraint is a hard limit. The moment a workload needs to spread across machines, Compose cannot help.

Kubernetes is the dominant production-grade orchestrator. Its design is declarative: you describe the desired system state and Kubernetes reconciles reality to match it continuously. This makes it inherently resilient — a crashed pod is just a discrepancy to be corrected. Horizontal Pod Autoscalers can add replicas when CPU usage crosses a threshold; rolling updates allow version upgrades with no downtime; per-container resource quotas prevent workloads from interfering with each other. The trade-off is operational complexity: Kubernetes has a steep setup and maintenance curve.

Docker Swarm is a pragmatic middle ground. It extends the Compose model to multi-node clusters with built-in load balancing, service replication, and automatic restart of failed tasks. The YAML format is largely compatible, which makes migration from a Compose setup straightforward. It offers less flexibility than Kubernetes but significantly less overhead. Apache Mesos takes a different architectural approach entirely — it functions as a distributed resource manager, pooling CPU and memory across a datacenter and offering slices to pluggable frameworks. Spark itself was designed to run on Mesos, which gives this combination particularly fine-grained resource isolation in large, multi-tenant environments.

Applied to the Task 2 Spark cluster, both Docker Swarm and Kubernetes represent meaningful improvements over the current setup. The immediate problem with using docker compose alone is the single point of failure: if the host goes down, the entire cluster goes with it. Docker Swarm resolves this with minimal configuration change — the existing YAML deploys as a distributed stack, the master and workers can run on separate nodes, and failed services restart automatically. Kubernetes goes further still: resource limits per container prevent the master and workers from competing for CPU, autoscaling can grow the worker pool during heavy jobs and shrink it afterward, and persistent volumes can preserve Spark event logs across restarts. For a production data pipeline, the investment in complexity is justified.

2 - Running a distributed system securely requires deliberate choices that are easy to skip when getting things working is the immediate priority. Looking at the Task 2 setup, several of those choices were not made.

The most direct risk is that both containers run as root. Docker's default behaviour when no USER directive is specified in the Dockerfile is to run everything as UID 0. Containers share the host kernel, so a process running as root inside the container has a shorter path to the underlying host if any container runtime vulnerability exists — either through a kernel exploit or a container escape bug. The fix is a single line in the Dockerfile: create a non-privileged user and switch to it before the CMD. It was not done here.

The second issue is network exposure. Port 7077, the Spark master's RPC endpoint, is bound to 0.0.0.0 and requires no authentication. Any process on the same Docker network can submit jobs to the cluster. Port 8080, the web UI, is equally open — it exposes submitted job history, executor details, environment variables, and configuration without credentials. In production these would be locked down: `spark.authenticate = true` for the RPC channel, TLS for transport encryption, and the web UI behind an authenticated reverse proxy restricted to internal networks.

To get beyond reasoning about this abstractly, I ran Trivy v0.69.3 against `sparkaiio/first:v0`. Trivy performs static analysis of OS packages and application library dependencies against the National Vulnerability Database and additional CVE sources. The image runs Ubuntu 22.04 and carries the full Spark 2.4.3 dependency tree, so there was significant surface area to cover.

The findings split into two categories. OS-level vulnerabilities across the base layer came to 35 — these are generally resolved by running package updates at build time. The JAR dependency findings were more substantive: `hadoop-common-2.7.3` had 7 CVEs, several of them deserialization vulnerabilities in network-accessible code paths; `commons-compress-1.8.1` had 6, including zip-bomb issues that could cause resource exhaustion; `avro-1.8.2` had 5. Further contributions came from `guava-14.0.1`, `commons-beanutils`, `hive-exec`, and `derby` — 2 to 3 CVEs each, with a number rated HIGH or CRITICAL under CVSS v3 scoring.

The root cause is straightforward: Spark 2.4.3 is a 2019 release and its dependency tree has not been updated since. For an isolated course environment this is workable. For anything carrying real data or exposed to a network, the remediation is clear: upgrade to a current Spark release, run containers as a non-root user, remove build tools from the final image, and integrate Trivy into the CI/CD pipeline so new vulnerabilities are caught before they reach production rather than after.